

Pilgrim Optimizer CLI: First Commands

Purpose

This guide explains the first three CLI commands in 'pilgrim-optimizer' and what they currently prove in the Ruleset A mancala sandbox.

The goal is to make the early loop explicit:

'scenario -> validate -> legal actions -> search -> recommendation'

Prerequisites

- Python 3.12+ installed ('python3 --version')
- Project dependencies installed from repository root:

```
python3 -m pip install -e ".[dev]"
```

- You are in the repository root:

```
cd /path/to/pilgrim-optimizer
```

Command 1: Validate a scenario

```
python3 -m pilgrim.cli validate scenarios/mancala_sandbox_001.json
```

What it does right now:

- Loads the scenario JSON file.
- Parses it and loads the linked setup/config JSON files.
- Checks simplified mancala-sandbox invariants.
- Confirms the scenario is a valid starting state for the current engine.

What it does ****not**** do:

- It does not solve the game.
- It does not prove full Pilgrim rules are implemented.

Typical output:

Scenario 'mancala_sandbox_001' is valid.

Command 2: List legal actions

```
python3 -m pilgrim.cli legal-actions scenarios/mancala_sandbox_001.json
```

What it does right now:

- Loads the same scenario.
- Asks the current rules engine to generate legal actions for the active player and phase.

- Prints stable, readable action IDs.

Why this matters:

- It is one of the fastest ways to debug action generation.
- It should usually be checked before trusting solver output.

Example output:

```
sow:0:1->2->3
```

```
sow:0:5->6->7
```

Command 3: Run the simple solver

```
python3 -m pilgrim.cli solve scenarios/mancala_sandbox_001.json --depth 3
```

What it does right now:

- Runs the current exact-search prototype.
- Searches to the specified depth ('--depth 3' in this example).
- Uses a temporary placeholder evaluation function.

Example output:

```
best_score=3
```

```
best_action=sow:0:1->2->3
```

```
nodes_expanded=27
```

How to interpret the current output

- 'best_score' is a value under the **temporary placeholder evaluation**, not true final Pilgrim VP.
- 'best_action=sow:0:1->2->3' means sow from city to north, north_east, east.
- 'nodes_expanded' is how many search nodes were explored.
- Together, this confirms the current development loop works end-to-end: 'scenario -> validate -> legal actions -> search -> recommendation'.

Position mapping used by the current sandbox:

- '0 = city'
- '1 = north'
- '2 = north_east'
- '3 = east'
- '4 = south_east'
- '5 = south'
- '6 = south_west'
- '7 = west'
- '8 = north_west'

What this does not mean yet

- The engine is not complete for full Pilgrim gameplay.
- The solver score is not final-game scoring quality.
- This is not strategic proof or balance validation.
- Only the current deterministic mancala-sandbox slice is covered.

Typical development workflow

1. Edit rules/config/scenario files.
2. Run 'validate' to ensure scenario integrity.
3. Run 'legal-actions' to inspect generated action space.
4. Run 'solve' with a small depth to sanity-check transitions/search loop.
5. Run 'pytest' to protect behavior with tests.

Troubleshooting

- 'zsh: command not found: python'
 - Use 'python3' instead of 'python'.
 - Optional alias:


```
'echo 'alias python=python3' >> ~/.zshrc && source ~/.zshrc'
```
- Scenario path errors
 - Confirm file exists: 'scenarios/mancala_sandbox_001.json'.
 - Run commands from repository root.
- Validation fails
 - Check JSON syntax and current invariants (acolyte conservation, non-negative resources, legal route lengths).
- Solver output seems surprising
 - Inspect 'legal-actions' first.
 - Remember the current scoring function is a temporary placeholder.

Next planned CLI improvements

- Clearer validation diagnostics (which invariant failed and why).
- Optional JSON output mode for tools/integration.
- Scenario diff helpers for fast debugging.
- Replay/event export commands for transition traces.
- Additional solver options beyond exact depth-limited search.